SatsPath Protocol v1

SatsPath is a payment discovery and routing protocol for Bitcoin rails. It defines signed payment profiles, verification rules, resolver semantics, and quote responses for Lightning, on-chain, and Ark receive pointers.

SatsPath is not a wallet, not a Bitcoin node, and not a transport-specific peer-to-peer system. Pear/Holepunch is one possible transport for moving SatsPath protocol objects between machines; it is not the protocol itself.

Scope

SatsPath v1 specifies:

- Human-readable recipient identifiers.
- Signed payment profiles.
- Public receive pointers for Lightning, on-chain Bitcoin, and Ark.
- Resolver semantics for locating signed profiles.
- Verification rules before routing.
- Quote response states for wallet and UI clients.
- Transport-neutral behavior, including how optional P2P transports carry protocol objects.

SatsPath v1 does not specify:

- Custody of funds.
- Wallet seed or spending-key storage.
- Mainnet transaction signing.
- Mainnet transaction broadcast.
- Lightning node operation.
- Ark server operation.

Payment execution belongs to the underlying wallet or rail. SatsPath discovers and validates where a payment can be sent, then returns public payment instructions.

Terminology

`Identifier` A human-readable name for a receiver, normally `user@domain`.

`PaymentProfile` Public profile data containing a receiver alias, protocol identity public key, supported payment methods, timestamps, and optional method-verification records.

`SignedPaymentProfile` A `PaymentProfile` plus a secp256k1 Schnorr signature made by the profile identity key.

`PaymentMethod` One public receive pointer. v1 methods are `Lightning`, `Onchain`, and `Ark`.

`Resolver` A component that maps an identifier to a `SignedPaymentProfile`.

`QuoteResponse` The protocol-level response returned after resolution, signature verification, expiry checks, route selection, and payment-payload construction.

`Transport` A way to carry SatsPath objects. Examples include local files, HTTPS, DNS/BIP-353, Nostr, and Pear/Holepunch.

Identifiers

The canonical identifier format is:

`<local-part>@<domain>`

Resolvers MUST treat identifiers as case-insensitive for lookup normalization unless the transport defines stricter semantics. Implementations SHOULD trim leading and trailing whitespace before lookup.

An identifier by itself is not an ownership proof. A profile signature proves control of the SatsPath protocol identity key embedded in the profile. DNSSEC, HTTPS well-known, Nostr/NIP-05, or a platform verifier

may add evidence that the identifier was published or approved by the relevant domain/account, but plain email syntax alone MUST be treated as `identifier-only`.

Examples:

```
alice@example.com
rodrigo@satspath.dev
shop@merchant.example
```

BIP-353 names MAY be represented with a leading Bitcoin sign:

```
₿alice@example.com
```

The leading ₿ sign selects DNS payment-instruction resolution behavior; it does not change the receiver identity once normalized.

Signed Payment Profiles

A `PaymentProfile` is public and contains no wallet secrets.

```
{
  "alias": "alice@example.com",
  "identity_pubkey": "02...",
  "methods": [
    {
      "type": "Lightning",
      "label": "Lightning Address",
      "lightning_address": "alice@example.com",
      "lnurl": null,
      "bolt12": null,
      "receiver_pubkey": null
    }
  ],
  "updated_at": 1782810000,
  "expires_at": null
}
```

A `SignedPaymentProfile` wraps the profile with a signature:

```
{
  "profile": {
    "alias": "alice@example.com",
    "identity_pubkey": "02...",
    "methods": []
  },
  "signature": "<128 lowercase hex characters>"
}
```

Signing Rule

The signature is computed over a domain-separated SHA-256 digest of the canonical JSON serialization of `profile`.

```
message = SHA256(UTF8("SatsPathProfileV1") || canonical_json_utf8(profile))
signature = secp256k1_schnorr_sign(identity_secret_key, message)
```

Implementations MUST keep serialization stable for the profile shape they sign. Implementations MUST NOT sign or transmit wallet seeds, xprv/tprv material, mnemonics, Lightning macaroons, Ark claim keys, refund keys, or other private spending material.

Verification Rule

Before using any profile for routing, an implementation MUST:

1. Parse `identity_pubkey` from hex as exactly a 33-byte compressed secp256k1 public key (`02`/`03` prefix).
2. Serialize with RFC 8259 JSON semantics, recursively lexicographically sorted object keys, UTF-8, and no insignificant whitespace.
3. Recompute `SHA256(UTF8("SatsPathProfileV1") || canonical_json_utf8(profile))`.
4. Decode `signature` as exactly 64 bytes of hex and verify secp256k1 Schnorr against the x-only projection of the compressed key. DER ECDSA MUST be rejected.
5. Reject the profile if `expires_at` is present and in the past.
6. Reject private material if any private-key-like content is detected in public fields.

Profile schema/signature version 1 is selected by `SatsPathProfileV1`; an incompatible future profile requires a new domain.

Key Rotation V1

The old key authorizes and the new key separately accepts these newline-delimited UTF-8 statements (no trailing newline):

```
SatsPathKeyRotationAuthorizationV1
identifier_hash
old_pubkey
new_pubkey
previous_event_hash
sequence
rotated_at
```

```
SatsPathKeyRotationAcceptanceV1
identifier_hash
old_pubkey
new_pubkey
previous_event_hash
sequence
rotated_at
```

Each complete statement is SHA-256 hashed and signed as a 64-byte secp256k1 Schnorr signature. `identifier_hash` is `SHA256(UTF8(canonical_identifier))`.

If signature verification fails, the protocol result MUST be `invalid_signature`.

If the profile is expired or unusable, the protocol result MUST be `no_route`.

Payment Methods

Lightning

Lightning methods carry at least one public Lightning receive pointer:

```
{
  "type": "Lightning",
  "label": "Lightning Address",
  "lightning_address": "alice@example.com",
  "lnurl": null,
  "bolt12": null,
  "receiver_pubkey": null
}
```

At least one of `lightning_address`, `lnurl`, or `bolt12` MUST be present for the method to be routable.

If a quote implementation can safely fetch a concrete BOLT11 invoice from an LNURL-pay endpoint, it MAY return that invoice as the payment payload. Otherwise it MUST return a public Lightning pointer.

On-chain

On-chain methods carry a public Bitcoin address:

```
{
  "type": "Onchain",
  "label": "Bitcoin (mainnet)",
  "network": "Mainnet",
  "address": "bc1q...",
  "pubkey_hint": null,
  "descriptor_hint": null
}
```

Implementations MUST validate that the address matches the expected network for the operating mode. Implementations SHOULD prefer fresh addresses and multiple on-chain methods for privacy, but v1 does not mandate address rotation.

Ark

Ark methods carry a public Ark server and receiver public key:

```
{
  "type": "Ark",
  "label": "Ark",
  "server": "https://ark.example.com",
  "pubkey": "02...",
  "vtxo_pointer": null,
  "proof": null,
  "expires_at": null
}
```

Ark receive pointers are public routing instructions. SatsPath v1 does not execute Ark transfers on mainnet.

Resolver Semantics

A resolver maps an identifier to a `SignedPaymentProfile`.

Resolver results are:

- `found`: a signed profile was found and returned.
- `not_found`: no signed profile exists for this identifier in that resolver.
- `unavailable`: resolver transport failed or timed out.
- `invalid`: resolver returned malformed or unsafe data.

A resolver chain MUST continue past `not_found` and transient `unavailable` results. A resolver chain MUST stop on a successfully verified profile. A resolver chain MAY stop on `invalid` when the invalid data is authoritative for that identifier.

Resolver behavior is specified in resolvers.md.

Quote Response Contract

SatsPath Mainnet Preview is a public-data-only protocol mode that operates strictly in the first three phases of a payment lifecycle:

1. **Resolve**: Fetch the signed profile for the identifier.
2. **Route**: Check signatures, expiry, ownership, and select the best rail.
3. **Invoice Presentation**: Fetch LNURL/BOLT11 data, format BIP-21 or Ark URIs, and display the payment pointer or QR.
4. **Execution**: **(Out of scope for Mainnet Preview)** The user scans the QR and pays using their own wallet.

The protocol quote response is serialized as a JSON object tagged by `status`.

`ok`

Returned when a profile is found, verified, not expired, and at least one route is available.

```
{
  "status": "ok",
  "recipient": {
    "alias": "alice@example.com",
    "verified": true,
    "profile_signature_verified": true,
    "identifier_verified": false,
    "identifier_verification": "identifier-only; no inbox/domain ownership proof in this response",
    "fingerprint": "a8fdac91"
  },
  "selected_method": {
    "type": "Lightning",
    "label": "Lightning Address",
    "lightning_address": "alice@example.com"
  },
  "fee_sats": 1,
  "eta": "instant",
  "reason": "Amount is below Lightning threshold and Lightning is available.",
  "qr": "alice@example.com"
}
```

The payload preview formats are:

- `lightning:<address>` for Lightning Address preview,
- `lnurl:<url>` for LNURL preview,
- a BOLT11 invoice string only when explicitly fetched by the caller,
- `bitcoin:<address>?amount=<btc>` for on-chain mainnet preview,
- `satspath:ark?...&network=mainnet` for Ark public pointer preview.

It must not:

- sign transactions,
- broadcast transactions,
- execute Lightning payments,
- execute Ark transfers,
- execute swaps,
- handle seeds, xprv/tprv, macaroons, certs, API secrets, claim keys, refund keys, or wallet private keys.

Mainnet execution (Phase 4 automation) is not part of v1 prototype behavior and no execution flag exists for mainnet.

Experimental Swap Engine (Execution Phase)

Execution automation (such as submarine swaps, reverse swaps, or Ark VTXO transfers) is built into an experimental engine that is **strictly testnet-only**. This engine handles Phase 4, but is disabled by default

and sandboxed away from mainnet to prevent any risk to real funds.

Invite Flow

When the resolver cannot find a registered alias, the sender creates an invite instead of failing silently:

```
{
  "invite": {
    "alias_hash": "a1b2c3d4...",
    "amount_sats": 1000,
    "created_at": 1782810000,
    "claim_url": "https://satspath.local/claim?alias_hash=...&amount=...",
    "warning": "Receiver must generate their own keys locally."
  }
}
```

`qr` is the public payment payload to show, copy, or hand off to a wallet.

`recipient.verified` is a legacy compatibility field. Clients SHOULD prefer the explicit `profile_signature_verified` and `identifier_verified` fields. It MUST NOT be displayed as email/inbox verification unless `identifier_verified` is also true.

`not_registered`

Returned when no resolver can find a signed profile.

```
{
  "status": "not_registered",
  "invite": {
    "alias_hash": "be8979...",
    "amount_sats": 250,
    "created_at": 1782810000,
    "claim_url": "https://satspath.local/claim?alias_hash=...&amount=250",
    "warning": "Receiver must generate their own keys locally."
  }
}
```

No funds should move solely because an invite exists. The receiver must publish a signed profile before a normal quote can be produced.

`no_route`

Returned when a profile is valid but no supported payment method can be selected.

```
{
  "status": "no_route",
  "reason": "No supported payment method is available."
}
```

`invalid_signature`

Returned when a profile is found but signature verification fails.

```
{
  "status": "invalid_signature",
  "recipient": {
    "alias": "alice@example.com",
    "verified": false,
```

```
    "fingerprint": null
  }
}
```

Clients MUST NOT offer payment execution for `invalid_signature`.

Routing Semantics

The v1 routing engine receives:

```
recipient identifier
amount_sats
verified SignedPaymentProfile
fee environment
```

The current reference policy is:

1. Prefer Lightning for small amounts when a Lightning method is available.
2. Consider on-chain when fee conditions make it acceptable and an on-chain method is available.
3. Consider Ark when an Ark method is available.
4. Return `no_route` when no available method passes policy.

The response MUST include a human-readable `reason` describing the selected route.

Routing policy MAY evolve without changing the quote response wire shape.

Mandatory Transparency Composition

The standalone daemon MUST NOT route `/v1/quote`, `/v1/pay` or `/v1/preview` from a self-contained signed profile alone. These endpoints use the same result as `/v1/resolve` and verify, in order: canonical profile signature and expiry, identifier history, key continuity, latest profile/event binding, inclusion in the exact signed checkpoint, consistency with the `log_id` pin, operator-key continuity, and payment-method ownership. Failure is terminal and MUST NOT fall back to the legacy resolver chain.

Name-event hashing uses two distinct values:

```
signing_payload_hash = SHA256("SatsPathNameEventPayloadV1" || canonical_json(unsigned_event))
owner_signature      = Schnorr("SatsPathNameEventSignatureV1\n" || hex(signing_payload_hash))
signed_event_hash    = SHA256("SatsPathSignedNameEventV1" || canonical_json(full_signed_event))
leaf_hash            = SHA256(0x00 || raw(signed_event_hash))
```

All quoted strings above are exact UTF-8 bytes with no NUL separator. Hex is lowercase. `previous_event_hash` is the preceding `signed_event_hash`. Registration uses sequence 0; every accepted mutation increments exactly once, and profile, event and rotation sequences MUST match.

An inclusion result is valid only if proof root/size equal checkpoint root/size, the leaf is derived from the requested signed event hash, the Merkle audit path verifies, and the checkpoint's 64-byte secp256k1 Schnorr signature verifies. Pins are keyed by stable `log_id`; a changed operator public key requires old-key authorization and new-key acceptance in a versioned operator rotation.

Payment Handoff

SatsPath v1 returns public payment payloads. It does not spend funds.

An implementation MAY expose a local API such as:

```
POST /v1/pay
```

This API MUST still return a protocol decision and a wallet handoff. It MUST NOT sign, broadcast, or custody funds unless a future protocol version explicitly defines an execution extension.

Transport Neutrality

SatsPath objects can move over multiple transports:

- Local registry files.
- HTTPS well-known endpoints.
- BIP-353 DNS payment instructions.
- Nostr/NIP-05 profile announcements.
- Pear/Holepunch peer transport.
- Manual file export/import.

All transports carry the same protocol objects and MUST be verified using the same profile verification rules.

Nostr Transport Profile

Nostr transport uses NIP-05 to bind an identifier domain to a Nostr pubkey and relay list. The SatsPath profile itself is carried in a replaceable Nostr event:

```
{
  "kind": 30078,
  "pubkey": "<nip05 nostr pubkey hex>",
  "tags": [["d", "satspath-profile:alice@example.com"]],
  "content": "{\"profile\":{...},\"signature\":\"3044...\"}"
}
```

Resolution MUST verify:

1. The NIP-05 document maps the local identifier part to the event author pubkey.
2. The event kind is `30078`.
3. The event author is the NIP-05 pubkey.
4. The `d` tag is `satspath-profile:<canonical identifier>`.
5. The event content parses as `SignedPaymentProfile`.
6. The SatsPath profile signature and expiry checks pass.

Nostr event signatures do not replace SatsPath profile signatures. Nostr is a discovery transport; SatsPath profile verification remains mandatory.

Pear/Holepunch is specified as an optional wire transport in wire_p2p.md.

Security Requirements

Implementations MUST:

- Verify signed profiles before routing.
- Reject malformed public keys and signatures.
- Reject expired profiles.
- Reject private material in public fields.
- Treat resolver output as untrusted until verified.
- Fail closed for DNSSEC-required flows unless explicitly placed into an insecure development mode.
- Never convert an invite into a payable route without re-resolving a signed profile.

Implementations SHOULD:

- Display whether a recipient was verified.
- Display the selected rail and routing reason.
- Separate route decision from payment execution.
- Keep resolver failures observable for debugging.

Reference Implementation Mapping

The current Rust implementation follows this spec through:

- `crates/satspath-core/src/profile.rs`: `PaymentProfile`, `PaymentMethod`, and `SignedPaymentProfile`.
- `crates/satspath-core/src/crypto.rs`: profile signing and verification.
- `crates/satspath-core/src/resolver.rs`: resolver trait and resolver chain semantics.
- `crates/satspath-core/src/resolvers/*`: transport-specific resolvers.
- `crates/satspath-router/src/quote_response.rs`: `QuoteResponse` contract and quote orchestration.
- `crates/satspath-router/src/router.rs`: route selection policy.
- `crates/satspathd/src/main.rs`: local daemon API exposing status, profile, peers, DNS resolution, quote, and pay handoff.